# Elements of AIML
# Unit 03 – Lecture 03 Notes
# Dimensionality Reduction (PCA, LDA, ICA)

Tofik Ali

April 9, 2026

## Contents

**1 Lecture Overview** **2**

**2 How to use these notes (self-study)** **2**

**3 Learning Outcomes** **2**

**4 Before you start** **2**

**5 Keywords** **2**

**6 Abbreviations and symbols** **3**

**7 Why dimensionality reduction?** **3**
7.1 Curse of dimensionality (intuition) . . . 3
7.2 When should you reduce dimensions? . . . 3
7.3 Dimensionality reduction vs feature selection . . . 3

**8 Principal Component Analysis (PCA)** **4**
8.1 Idea . . . 4
8.2 What is being optimized? (important intuition) . . . 4
8.3 Steps (more detail) . . . 4
8.4 How to get eigenvalues and eigenvectors manually . . . 4
8.5 Worked 2D example: manual eigenvalues and eigenvectors . . . 5
8.6 Explained variance . . . 6
8.7 How to choose $k$ in practice . . . 6
8.8 Why standardize? . . . 6
8.9 A tiny geometric example (2D $\rightarrow$ 1D) . . . 7
8.10 PCA and SVD (brief note) . . . 7
8.11 Common pitfalls . . . 7

**9 LDA and ICA (conceptual preview)** **7**
9.1 LDA . . . 7
9.2 ICA . . . 7

**10 Python example (PCA)** **8**

**11 Practice (with answers)** **8**

**12 Exit questions (with answers)** **8**

**13 Summary** **9**

# 1 Lecture Overview

This lecture introduces **dimensionality reduction** and focuses on PCA, with conceptual overviews of LDA and ICA. Dimensionality reduction is important for:

- faster training/inference,
- reducing noise and redundancy,
- visualization of high-dimensional datasets.

*Sources:* Mueller and Massaron, *Machine Learning for Dummies*, For Dummies, 2016.;Mitchell, *Machine Learning*, selected chapters on learning basics and evaluation.;Scikit-learn documentation, accessed 2026-02-28.

# 2 How to use these notes (self-study)

First, build intuition: PCA finds directions of maximum variance and projects data onto them. Then study the step-by-step pipeline (centering, covariance, eigenvectors, projection) and work through the practice questions. When using PCA in code, remember the evaluation rule: fit PCA on training data only to avoid leakage.

# 3 Learning Outcomes

- Explain the curse of dimensionality at an intuitive level.
- Describe PCA and interpret principal components and explained variance.
- Outline PCA steps and apply PCA using Python.
- Distinguish PCA vs LDA vs ICA conceptually.

# 4 Before you start

This lecture assumes two earlier Unit III ideas are already clear:

- from **Unit 03 Lecture 01**: features are design choices, and their scale/representation affects the model,
- from **Unit 03 Lecture 02**: preprocessing must be fit on the **training split only** to avoid leakage.

PCA is a feature-transformation method, so both ideas still apply.

# 5 Keywords

- **Dimensionality:** number of features ($d$).

- **Principal component:** direction capturing maximum variance, orthogonal to previous.
- **Explained variance:** variance captured by a component (often ratio).
- **Projection:** mapping data to a lower-dimensional space.
- **Covariance matrix:** captures relationships between features.

# 6 Abbreviations and symbols

- $n$: number of samples (rows), $d$: original number of features, $k$: reduced number of components.
- $X \in \mathbb{R}^{n \times d}$: data matrix; $X_c$: mean-centered data.
- $C$: covariance matrix; $\lambda_i$: eigenvalues; $v_i$: eigenvectors.
- $V_k$: matrix of top-$k$ eigenvectors; $Z = X_c V_k$: projected data.
- **EVR**: explained variance ratio $r_i = \lambda_i / \sum_j \lambda_j$.

# 7 Why dimensionality reduction?

## 7.1 Curse of dimensionality (intuition)

As dimensionality increases:

- data becomes sparse (needs more samples to cover space),
- distances can become less informative,
- models can overfit more easily.

Reducing dimensions can improve generalization and computational efficiency.

## 7.2 When should you reduce dimensions?

Common reasons:

- **Speed**: fewer features $\Rightarrow$ faster training and prediction.
- **Noise reduction**: remove low-variance directions that often contain noise.
- **Visualization**: project to 2D/3D for plots and intuition.
- **Redundancy**: features may be highly correlated; PCA compresses them.

## 7.3 Dimensionality reduction vs feature selection

- **Feature selection**: keep a subset of original features (easier to interpret).
- **Feature extraction**: create new features (components) from combinations of originals (e.g., PCA).

# 8 Principal Component Analysis (PCA)

## 8.1 Idea

PCA finds a new coordinate system where:

- the first axis captures the maximum variance,
- the second captures the next maximum variance while being orthogonal, etc.

Then we keep only the top $k$ axes and drop the rest.

## 8.2 What is being optimized? (important intuition)

PCA can be motivated in two equivalent ways:

- **Max-variance view:** choose a unit vector $v$ that maximizes the variance of the projection $X_c v$.
- **Min-reconstruction-error view:** choose a $k$-dimensional subspace that minimizes squared reconstruction error.

Both lead to the same principal components.

## 8.3 Steps (more detail)

Given data matrix $X$ with $n$ samples and $d$ features:

1. (Often) standardize features: subtract mean and divide by std.
2. Mean-center: $X_c = X - \mu$ (where $\mu$ is the mean of each feature).
3. Compute covariance matrix:
$$C = \frac{1}{n-1} X_c^\top X_c$$
4. Compute eigenvalues $\lambda_i$ and eigenvectors $v_i$ of $C$.
5. Sort eigenvectors by decreasing eigenvalues.
6. Choose top-$k$ eigenvectors and project: $Z = X_c V_k$.

## 8.4 How to get eigenvalues and eigenvectors manually

For PCA, you do **not** find eigenvalues directly from the raw data matrix. You first compute the covariance matrix $C$, and then solve:
$$Cv = \lambda v$$
where:

- $\lambda$ is an eigenvalue, which tells how much variance that direction captures,
- $v$ is an eigenvector, which gives the direction of that component.

For a small matrix, the manual steps are:

1. Form $C$ from the centered data.

2. Solve the characteristic equation:
$$\det(C - \lambda I) = 0$$
to obtain the eigenvalues.

3. For each eigenvalue $\lambda$, solve:
$$(C - \lambda I)v = 0$$
to obtain the corresponding eigenvector.

4. Normalize the eigenvector to unit length.

5. Sort eigenvalues in decreasing order; the eigenvector of the largest eigenvalue is the first principal component.

This is practical only for tiny examples such as $2 \times 2$ covariance matrices. Real software uses more stable matrix methods internally.

## 8.5 Worked 2D example: manual eigenvalues and eigenvectors

Suppose that after centering the data, the covariance matrix is:

$$C = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

**Step 1: Find eigenvalues.** Solve:

$$\det(C - \lambda I) = \det \begin{bmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{bmatrix} = 0$$

So:

$$(3-\lambda)^2 - 1 = 0$$
$$9 - 6\lambda + \lambda^2 - 1 = 0$$
$$\lambda^2 - 6\lambda + 8 = 0$$
$$(\lambda - 4)(\lambda - 2) = 0$$

Hence the eigenvalues are:

$$\lambda_1 = 4, \qquad \lambda_2 = 2$$

**Step 2: Find eigenvector for $\lambda_1 = 4$.**

$$C - 4I = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}$$

Solve:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0$$

This gives $-x + y = 0$, so $y = x$. One eigenvector is therefore:

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Normalized:

$$\hat{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

**Step 3: Find eigenvector for $\lambda_2 = 2$.**

$$C - 2I = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Solve:

$$x + y = 0 \Rightarrow y = -x$$

One eigenvector is:

$$v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Normalized:

$$\hat{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

**Interpretation for PCA.** Because $\lambda_1 = 4$ is larger than $\lambda_2 = 2$, the first principal component is the direction:

$$\hat{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

This is the diagonal direction. PCA keeps this direction first because it captures more variance than the perpendicular direction.

## 8.6 Explained variance

Eigenvalue $\lambda_i$ corresponds to variance captured by component $i$. Explained variance ratio:

$$r_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

We often choose $k$ such that $\sum_{i=1}^{k} r_i$ exceeds a threshold (e.g., 0.90), depending on the application.

## 8.7 How to choose $k$ in practice

Three common approaches:

- **Cumulative EVR**: pick smallest $k$ reaching 90%, 95%, etc.
- **Scree plot (elbow)**: choose $k$ where additional components add little.
- **Downstream performance**: treat $k$ as a hyperparameter and validate it (careful: do this inside CV to avoid leakage).

## 8.8 Why standardize?

If features have different scales (e.g., income in thousands vs age in years), PCA may focus on the largest-scale feature. Standardization makes PCA reflect correlation structure rather than raw scale.

### 8.9 A tiny geometric example (2D → 1D)

Suppose (after centering) the points lie roughly on the line $x_2 \approx x_1$. Then the first principal component is approximately:

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Projecting a point $x$ to 1D gives the scalar coordinate:

$$z = x^\top v_1$$

Intuition: we keep the "diagonal direction" and drop the perpendicular direction, losing little information if points are tightly clustered around the diagonal.

### 8.10 PCA and SVD (brief note)

Many software libraries compute PCA using **SVD** (singular value decomposition) for numerical stability. You do not need to compute SVD by hand, but it explains why PCA works well for large matrices.

### 8.11 Common pitfalls

- PCA is **linear**: it cannot capture complex non-linear structure (for that, methods like t-SNE/UMAP exist).
- Components can be hard to interpret; they are combinations of features.
- Always fit PCA on **training data only** (otherwise you leak information from validation/test).

## 9 LDA and ICA (conceptual preview)

### 9.1 LDA

Linear Discriminant Analysis is supervised: it uses class labels and tries to find a projection that separates classes.

**How it differs from PCA.** PCA ignores labels and maximizes variance overall. LDA uses labels and tries to maximize between-class separation while minimizing within-class spread.

**Why it is only a preview here.** LDA makes much more sense after you have seen classification properly in **Unit 04 Lecture 02**. For now, remember only this anchor: PCA ignores labels, while LDA uses them.

### 9.2 ICA

Independent Component Analysis tries to find components that are statistically independent. A classic intuition is separating mixed signals into original sources.

**Where ICA is used.** Blind source separation (signals), artifact removal, and some feature extraction tasks. ICA assumes the underlying sources are independent and non-Gaussian.

## 10 Python example (PCA)

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2))
])

Z_train = pipe.fit_transform(X_train)
Z_test = pipe.transform(X_test)
print(pipe.named_steps["pca"].explained_variance_ratio_)
```

The key rule is unchanged from the previous lecture: fit scaling and PCA on **training data only**, then reuse the learned transformation on validation/test data.

## 11 Practice (with answers)

### P1. Why is it incorrect to apply PCA on the full dataset before splitting into train/test?

**Answer:** PCA learns directions from data. If you fit PCA using test data, information from test influences the representation used in training, causing leakage and optimistic evaluation.

### P2. PCA vs feature selection: which is easier to interpret and why?

**Answer:** Feature selection is easier to interpret because it keeps original features. PCA creates components that mix features, which can be harder to explain.

### P3. If EVR values are $[0.60, 0.25, 0.10, 0.05]$, what is cumulative EVR for $k = 2$?

**Answer:** $0.60 + 0.25 = 0.85$ (85%).

## 12 Exit questions (with answers)

### Q1. What does PCA optimize?

**Answer:** PCA chooses orthogonal directions that maximize variance of the projected data (sequentially), thus capturing as much variability as possible in fewer dimensions.

### Q2. Why standardize before PCA?

**Answer:** to avoid features with larger numeric scales dominating the covariance and the principal components.

### Q3. Compare PCA and LDA.

**Answer:** PCA is unsupervised and maximizes variance; LDA is supervised and aims to separate classes by maximizing between-class separation relative to within-class scatter.

### Q4. What is explained variance?

**Answer:** the amount (or proportion) of total variance in the data captured by a principal component; used to choose the number of components.

## 13 Summary

PCA is a core dimensionality reduction tool. LDA and ICA provide alternative projections depending on whether labels are available (LDA) or independence is desired (ICA).

*Sources:* Mueller and Massaron, *Machine Learning for Dummies*, For Dummies, 2016.;Mitchell, *Machine Learning*, selected chapters on learning basics and evaluation.;Scikit-learn documentation, accessed 2026-02-28.